point should be hidden.

In general, be careful when using `has_no_field?` as there are an uncountable number of fields the form doesn't have, and any typo will cause such a test to pass. In this case we're safe, as the test contains matching `has_field?` method calls.

After that, we `select` the value `"Check"` from the `"Pay type"` selector and then assert that the routing number text field showed up, using `has_field`. We repeat this for each combination of Pay type and field. Four groups of five assertions, for a total of twenty asssertions. Whew!

Capybara makes all of this possible using a compact, readable API that requires very little code. For additional information and more methods, we suggest that you familiarize yourself with the domain-specific language (DSL) that Capybara provides.[61]

Now let's run the test we just wrote:

```
$ bin/rails test:system
Running 5 tests in a single process (parallelization threshold is 50)
Run options: --seed 55897

# Running:

Capybara starting Puma...
* Version 5.5.2 , codename: Zawgyi
* Min threads: 0, max threads: 4
* Listening on tcp://127.0.0.1:56776
Capybara starting Puma...
* Version 3.12.1 , codename: Llamas in Pajamas
* Min threads: 0, max threads: 4
* Listening on tcp://127.0.0.1:43749
.....

Finished in 4.065668s, 1.2298 runs/s, 5.9031 assertions/s.

5 runs, 24 assertions, 0 failures, 0 errors, 0 skips
```

When you run this, you'll notice a number of things. First, a web server is started on your behalf, and then a browser is launched and the actions you requested are